

Python für Java-Lernende – Kurzfassung

Python ist ähnlich wie Java, braucht aber weniger Schreibarbeit: keine Typangabe bei jeder Variable, keine geschweiften Klammern und meist kein Semikolon. Einrückung gehört zur Syntax.

1. Variablen und Ausgabe

Java: `int x = 5`; Python erkennt den Typ automatisch. Strings kannst du mit f-Strings formatieren.

```
x = 5
name = "Ada"
print(f"{name}: {x}")
```

2. if / elif / else

Wie Java `if/else if/else`, aber mit Doppelpunkt und Einrückung.

```
alter = 18
if alter >= 18:
    print("Volljährig")
elif alter >= 16:
    print("Fast volljährig")
else:
    print("Minderjährig")
```

3. Schleifen, break und pass

`for` läuft über Elemente oder `range()`; `while` läuft, solange die Bedingung wahr ist. `break` beendet die Schleife; `pass` macht nichts.

```
for i in range(1, 6): # 1 bis 5
    if i == 4:
        break
    print(i)
while False:
    pass
```

4. Listen und Funktionen

Listen ähneln `ArrayLists`; `def` definiert eine Funktion.

```
zahlen = [2, 4, 6]
zahlen.append(8)
def quadrat(n):
    return n * n
print(quadrat(zahlen[0])) # 4
```

5. Fehlerbehandlung

`try/except` entspricht grob `try/catch` in Java.

```
try:
    zahl = int("abc")
except ValueError:
    print("Keine gültige Zahl")
```

6. Bedingter Ausdruck und match-case

Kurzform für `if/else`; `match-case` ist ähnlich wie `switch` in Java (ab Python 3.10).

```
status = "ok" if x > 0 else "nicht ok"
match x:
    case 1:
        print("eins")
    case _:
        print("anderer Wert")
```

7. Wichtigste Unterschiede

Python verwendet and/or/not statt &&/||/!, True/False/None statt true/false/null und == für Wertvergleich. Typen sind dynamisch, aber Typ-Hinweise sind möglich.

```
def addiere(a: int, b: int) -> int:
    return a + b
```